StockIQ
Informe final del Trabajo Final Integrador

Universidad: Universidad Tecnológica Nacional (UTN)

Carrera: Tecnicatura Universitaria en Programación

Materia: Proyecto Final (Trabajo Final Integrador)

Proyecto: StockIQ - Gestión de inventario multidepósito con reposición inteligente

Grupo: 183

Integrantes: Jennifer Mariel Franco y Jonathan Nahuel Franco

Tutora: Sofia Raia

Año: 2026

Repositorio: github.com/Yonathannf/TPI_FINAL_UTN_StockIQ

Sistema publicado: https://tpi-final-utn-stock-iq.vercel.app/

API publicada: stockiq-backend-peal.onrender.com

Video de presentación: https://youtu.be/QdCD1V_JVE4

Indice

StockIQ
Informe final del Trabajo Final Integrador **0**
1. Problema y contexto 2
2. Objetivos y alcance 2
3. Análisis y modelo 3
4. Metodología de trabajo 4
5. Arquitectura y tecnologías 4
6. Implementación 6
Recorridos principales 6
7. Decisiones técnicas y dificultades resueltas 7
7.1 El contrato de la API como acuerdo entre los dos 7
7.2 Fechas y zona horaria 7
7.3 El criterio de corte del ABC 8
7.4 Operaciones simultáneas 8
7.5 Quién confirma una transferencia 8
7.6 La API no acepta campos de más 8
7.7 Ajustes pedidos en la revisión del diseño 8
7.8 Git en una carpeta sincronizada 8
8. Capa analítica 9
9. Pruebas y validación 9
10. Despliegue 12
11. Resultados, conclusiones y trabajo futuro 13

1. Problema y contexto

En muchos depósitos el stock se controla con planillas sueltas y el momento de volver a comprar se decide "a ojo". El resultado son quiebres de stock inesperados, plata inmovilizada en productos que casi no se venden y ninguna trazabilidad cuando la mercadería pasa de un depósito a otro.

El caso real. El problema surge de la experiencia directa de los dos integrantes. Jonathan es jefe de almacén en una pyme, donde el stock se controla con una planilla de Excel. Jennifer trabaja en el área de Recursos Humanos de una distribuidora, en una posición muy cercana a logística, desde donde conoce los problemas que genera el control de stock en el día a día. Las dos experiencias coinciden en las mismas limitaciones: el stock se actualiza a mano y depende de que alguien recuerde cargar cada movimiento; la planilla guarda el saldo, pero no quién movió qué, cuándo ni por qué; cuando la mercadería pasa de un lugar a otro, no hay forma de saber qué está en camino; y la decisión de volver a comprar se toma por experiencia, sin un cálculo que la respalde. La necesidad concreta fue reemplazar esa planilla por un sistema que registre cada movimiento y proponga cuándo y cuánto reponer.

En el almacén de Jonathan se manejan entre 600 y 700 productos. Con un catálogo de ese tamaño, en una planilla no hay forma práctica de identificar cuáles rotan más, cuáles rotan poco y cuáles están directamente paralizados, y por lo tanto tampoco de tomar medidas sobre ellos. Además, cuando llega mercadería de fábrica, el detalle viene en papel y hay que cargarlo a mano en la planilla, con el riesgo de cometer errores que nadie detecta después.

StockIQ reemplaza ese criterio manual por uno basado en datos. Cada entrada, salida y transferencia queda registrada con su fecha, su motivo y su responsable, y el stock nunca se escribe a mano: se calcula siempre a partir de esos movimientos. Sobre esa base, el sistema propone cuándo y cuánto reponer de cada producto, mide qué tan rápido rota el inventario y clasifica el catálogo según su impacto económico.

La diferencia con un ABM de stock tradicional está en esa capa analítica. Un sistema común avisa cuando el stock cae por debajo de un mínimo que alguien cargó; StockIQ calcula ese mínimo con el consumo real y el tiempo de entrega de cada proveedor.

2. Objetivos y alcance

El objetivo general fue desarrollar un sistema web para gestionar el inventario de uno o más depósitos, controlar los movimientos y las transferencias, y generar alertas y recomendaciones de reposición basadas en el consumo real.

Los objetivos específicos fueron:

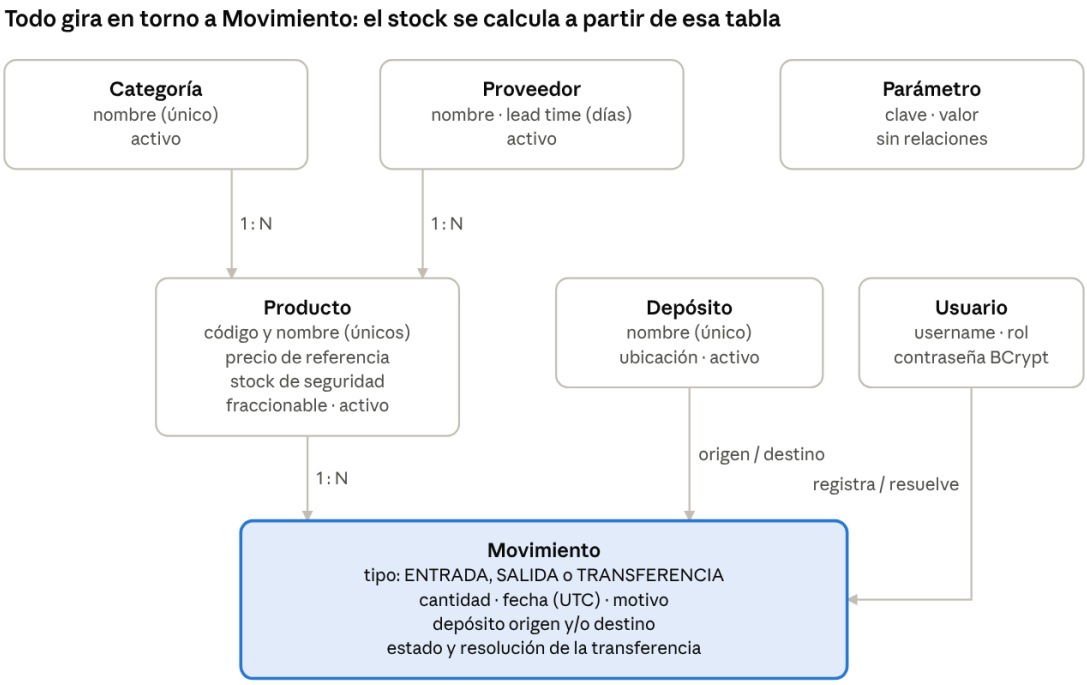
- Centralizar los productos, las categorías, los proveedores y los depósitos.
- Registrar entradas, salidas y transferencias con trazabilidad completa.
- Calcular el stock actual y el stock en tránsito a partir del historial de movimientos.
- Calcular el punto de reposición con el consumo promedio, el tiempo de entrega del proveedor y un stock de seguridad.
- Medir la rotación por producto y por categoría, y clasificar el catálogo con el análisis ABC.
- Mostrar los indicadores en un dashboard con alertas.

El sistema tiene dos roles. El **encargado** administra el catálogo, los depósitos, los usuarios y los parámetros, y es el único que ve la capa analítica. El **operario** registra movimientos, confirma o cancela transferencias y consulta el stock. La gerencia usa el rol de encargado para consultar el dashboard.

Quedaron fuera del alcance del MVP las devoluciones, los permisos por depósito, la integración con sistemas de facturación, las notificaciones por correo, el mapa del depósito y el cálculo del ABC y del punto de reposición por depósito (se calculan a nivel global). El detalle está en `docs/01-requerimientos.md`.

3. Análisis y modelo

El dominio se resume en siete entidades. Seis describen "qué hay" (el catálogo, los lugares, las personas y la configuración) y una describe "qué pasó": el movimiento, que es el centro del modelo.



modelo de datos · entidades principales y sus relaciones

Cada producto pertenece a una categoría y tiene un proveedor, cuyo tiempo de entrega (lead time) usa el cálculo de reposición. Cada movimiento registra un producto, una cantidad, quién lo hizo y cuándo; según su tipo, tiene un depósito de destino (entrada), de origen (salida) o los dos (transferencia). El stock por depósito y el stock en tránsito no son tablas: son dos vistas de la base que suman los movimientos. El diccionario completo de datos está en `docs/03-diseno-base-de-datos.md`.

Decisiones de modelado más importantes

- **El stock se deriva, no se guarda (D-01).** Evita que el saldo y el historial se contradigan, y deja toda la historia disponible para la capa analítica.

- **La transferencia es un solo movimiento con estado (pendiente, confirmada o cancelada).** Así el stock en tránsito se calcula sin tablas extra, y queda registrado quién resolvió cada transferencia y cuándo.
- **Los movimientos no se editan ni se borran (RN-07).** Un error se corrige con un movimiento nuevo, lo que preserva la trazabilidad.
- **Bajas lógicas.** Productos, categorías, proveedores, depósitos y usuarios se desactivan en lugar de borrarse, para no romper el historial.
- **Productos fraccionables o no.** Un campo indica si el producto admite cantidades con decimales (por ejemplo, kilos) o solo enteras (unidades, packs).

Reglas de negocio más relevantes (el listado completo, de RN-01 a RN-47, está en `docs/02-reglas-de-negocio.md`)

- Ninguna salida ni transferencia puede dejar stock negativo en el depósito de origen (RN-06).
- La mercadería de una transferencia sale del origen al registrarla y suma al destino solo cuando alguien confirma que llegó; si se cancela, vuelve al origen (RN-10 a RN-12).
- Una transferencia ya resuelta no se puede volver a confirmar ni cancelar (RN-13).
- La fecha de cada movimiento la asigna el servidor, en UTC (RN-41).
- Un depósito o un producto con stock no se puede dar de baja (RN-38, RN-39).
- Siempre tiene que quedar al menos un encargado activo (RN-40).
- El operario no accede a la capa analítica ni a la administración (RN-34, RN-35).

4. Metodología de trabajo

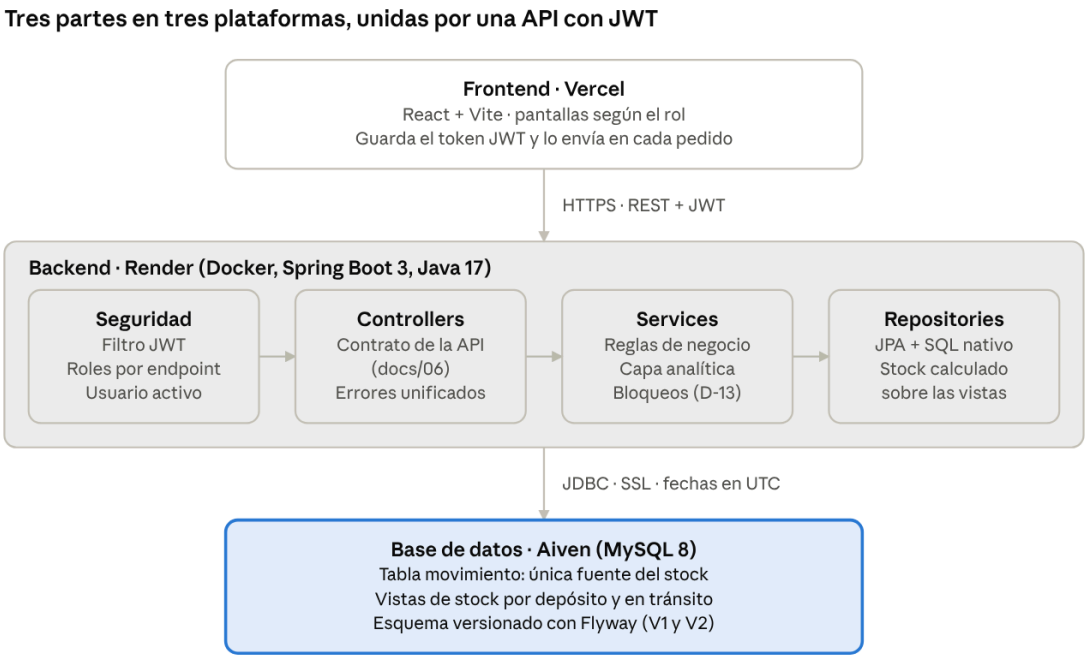
Trabajamos en etapas semanales, cada una con un objetivo cerrado, y subimos el código por partes a un único repositorio en GitHub. Antes de codificar cerramos el análisis: la tutora aprobó en octubre de 2026 los requerimientos, las reglas de negocio, el diseño de la base de datos y el listado de módulos, lo que nos dio la condición de regular.

Cada parte se desarrolló en su propia rama (`feature/...` para funcionalidades nuevas, `fix/...` para correcciones y `docs/...` para documentación) y entró a `main` solo a través de un pull request revisado por el otro integrante. Los mensajes de commit siguen una convención por tipo (`feat:`, `fix:`, `docs:`, `test:`), lo que permite reconstruir el avance desde el historial del repositorio.

Nos repartimos el trabajo por capas en cada etapa: mientras uno hacía el backend de un módulo, el otro hacía sus pantallas, y en la etapa siguiente invertíamos los roles. Así los dos trabajamos sobre las dos capas del sistema. Para que ese trabajo en paralelo funcionara, el contrato de la API (`docs/06-api-endpoints.md`) terminó siendo el punto de acuerdo entre los dos: define, para cada endpoint, el JSON exacto que se manda y el que se recibe. En la sección 7 explicamos por qué llegamos a esa forma de trabajar.

5. Arquitectura y tecnologías

StockIQ tiene tres partes publicadas en tres plataformas: el frontend en React, el backend en Spring Boot organizado en capas y la base de datos MySQL. El frontend nunca accede a la base: todo pasa por la API, que valida la sesión y el rol en cada pedido.



arquitectura · frontend, backend en capas y base de datos

La decisión de diseño central es que **el stock no se guarda**: se calcula siempre a partir de la tabla de movimientos (D-01). Eso garantiza que lo que muestra el sistema coincide con lo registrado y deja todo el historial disponible para la capa analítica.

El backend sigue la arquitectura Controller → Service → Repository, con DTOs y mappers, y está dividido en un paquete por módulo: `config`, `security`, `common`, `usuario`, `catalogo`, `deposito`, `movimiento`, `stock`, `parametro`, `analitica` y `dashboard`. Las tecnologías principales son:

- **Java 17 y Spring Boot 3 (backend).** Es el stack que trabajamos en la carrera, y Spring resuelve de forma probada lo que el sistema necesita: transacciones (cada movimiento se registra completo o no se registra), seguridad y validaciones. Spring Data JPA con Hibernate maneja las entidades, y para los cálculos analíticos usamos SQL directo, que es más claro y más rápido para agregar miles de movimientos.
- **MySQL 8 (base de datos relacional).** El dominio es relacional por naturaleza (productos, depósitos, usuarios y movimientos que se referencian entre sí) y necesita integridad: claves foráneas, restricciones y transacciones con bloqueo de filas para las operaciones simultáneas. Una base documental no hubiera garantizado esas reglas.
- **Flyway (migraciones).** Versiona el esquema: la base se reconstruye igual en cualquier entorno, y los cambios quedan registrados (por ejemplo, la migración V2 con los ajustes pedidos en la revisión del diseño). Hibernate solo valida que las entidades coincidan con las tablas; nunca modifica el esquema (D-14).
- **Spring Security con JWT y BCrypt (autenticación y autorización).** El token JWT permite una API sin sesiones en el servidor, adecuada para un frontend separado. Las contraseñas se guardan cifradas con BCrypt, y los permisos por rol se controlan en el backend en cada pedido, no solo

ocultando botones. Además, el backend comprueba en cada pedido que el usuario siga activo, para que una baja corte el acceso al instante (RF-02).

- **React con Vite (frontend).** Permite armar pantallas por componentes reutilizables, con una compilación rápida. Se usan React Router para las rutas protegidas por rol, Axios para hablar con la API, Lucide para los íconos y Recharts para los gráficos del dashboard.
- **Docker y docker-compose.** Empaquetan el backend para Render y permiten levantar el backend y MySQL en local con un solo comando, igual para los dos integrantes.
- **JUnit 5, Mockito, Testcontainers, MockMvc y JaCoCo (pruebas).** Pruebas unitarias rápidas para las reglas, pruebas de integración contra un MySQL real para las consultas y los bloqueos, y un reporte que mide la cobertura.

6. Implementación

El desarrollo llevó cinco etapas semanales, entre el 12 de octubre y el 15 de noviembre de 2026, y terminó con más de doce pull requests integrados en `main`. La tabla resume qué se construyó en cada una.

Etapa	Backend	Frontend
1. Base (12 al 18/10)	Configuración, manejo de errores unificado, usuarios, login con JWT y seguridad por rol	Proyecto Vite + React, login, rutas protegidas, menú por rol y rediseño visual a partir de una maqueta en Canva
2. Datos maestros (19 al 25/10)	ABM de categorías, proveedores, productos y depósitos con las reglas RN-30 a RN-47	Pantallas de catálogo, depósitos y usuarios
3. Núcleo (26/10 al 1/11)	Movimientos, transferencias con estados, historial paginado y cálculo de stock	Registro de movimientos, historial con filtros, transferencias pendientes y stock por depósito
4. Capa analítica (2 al 8/11)	Parámetros, punto de reposición, rotación, ABC y endpoints del dashboard	Dashboard y configuración de parámetros
5. Cierre (9 al 15/11)	Datos de demostración, tests de la capa analítica y ajuste de RF-12	Pruebas de punta a punta en producción

El despliegue no quedó para el final. Siguiendo una recomendación de la tutora, lo hicimos con el sistema a medio construir, para encontrar temprano los problemas de configuración (sección 10). Desde entonces, cada pull request integrado en `main` se publica solo.

Recorridos principales

Estos tres casos de uso muestran cómo trabajan juntas las partes del sistema.

Transferencia entre depósitos. Un operario registra que manda 6 cajas de cerveza del depósito Central a la Sucursal Norte. Al registrarla, el backend bloquea el producto, verifica que haya stock suficiente en el

origen y guarda el movimiento como pendiente: la mercadería deja de contarse en Central y aparece "en tránsito". Cuando llega, alguien de la sucursal confirma la recepción y el stock suma en el destino; si se cancela, vuelve al origen. En los dos casos queda registrado quién la resolvió y cuándo.

- ☐ Captura: formulario de transferencia y tabla de stock con la columna "En tránsito".

Alerta de reposición. El encargado abre el dashboard y ve las alertas. Al desplegar un producto, el sistema muestra la cuenta completa: consumo diario de los últimos 30 días × tiempo de entrega del proveedor + stock de seguridad = punto de reposición, y la cantidad sugerida para cubrir los próximos 30 días, descontando lo que ya está en camino.

- ☐ Captura: alerta de reposición con el detalle del cálculo desplegado.

Clasificación ABC y capital inmovilizado. El dashboard muestra la curva de Pareto con los cortes del 80 % y el 95 %, el reparto de productos y de valor por clase, y los productos de menor rotación con el dinero que tienen inmovilizado en stock.

- ☐ Captura: curva de Pareto y sección de baja rotación del dashboard.

7. Decisiones técnicas y dificultades resueltas

Los problemas más importantes del desarrollo no fueron de programación sino de acuerdos: entre el frontend y el backend, entre el código y la documentación, y entre lo que se planificó y lo que ofrecían las plataformas. Estos son los casos que más nos enseñaron.

7.1 El contrato de la API como acuerdo entre los dos

En la segunda etapa, la pantalla de productos mostraba las columnas de categoría y proveedor vacías, y al editar un producto no dejaba guardar. El backend devolvía la categoría y el proveedor anidados (`categoria: { id, nombre, activo }`), mientras que el frontend los buscaba como campos sueltos (`categoriaId`, `categoriaNombre`). Cada uno había interpretado distinto un documento que listaba los endpoints pero no la forma de sus respuestas.

La corrección llevó seis líneas. Lo importante fue el cambio de método: desde entonces, antes de programar un módulo escribimos en `docs/06-api-endpoints.md` el JSON exacto de cada pedido y cada respuesta. Con movimientos y stock, el frontend se hizo antes que el backend, contra ese contrato, y al integrarlos funcionaron a la primera.

7.2 Fechas y zona horaria

La tutora nos pidió definir quién asigna la fecha de un movimiento y en qué zona horaria se calculan las ventanas del análisis, porque los servidores en la nube trabajan en UTC. Decidimos que la fecha la asigna siempre el servidor (RN-41) y que todo se guarda en UTC, garantizado en tres niveles: la máquina virtual de Java, Hibernate y la sesión de MySQL (`forceConnectionTimeZoneToSession=true`).

La conversión a la hora de Argentina se hace solo en los bordes: al mostrar las fechas en pantalla, al interpretar los filtros por día del historial y al agrupar por día en el dashboard. Un movimiento de las 22:30 del 2 de octubre en Argentina se guarda como 01:30 del 3 de octubre en UTC, y gracias a esa conversión igual aparece en el día correcto. Además, las consultas analíticas no usan `NOW()`: reciben la fecha de referencia desde un reloj de la aplicación, lo que permite probarlas con una fecha fija.

7.3 El criterio de corte del ABC

Al escribir la consulta del ABC apareció una ambigüedad: ¿en qué clase queda el producto que cruza el 80 %? Con el criterio más directo (comparar el acumulado incluyendo al propio producto), un catálogo con un solo producto quedaba en clase C, lo que contradecía nuestra propia regla RN-28. Elegimos comparar el porcentaje acumulado **previo** al producto (D-12): así el producto que cruza un corte queda en la clase superior, y el de mayor valor siempre es A. Es también el criterio habitual del análisis de Pareto.

7.4 Operaciones simultáneas

Como el stock no se guarda sino que se calcula, dos salidas registradas al mismo tiempo podrían leer el mismo stock y dejarlo negativo. Lo resolvimos bloqueando la fila del producto (`SELECT ... FOR UPDATE`) antes de validar (D-13). Aplicamos el mismo criterio a la regla que impide quedarse sin encargados (RN-40): sin bloqueo, dos encargados que se desactivaran entre sí al mismo tiempo dejarían el sistema sin administrador.

7.5 Quién confirma una transferencia

Al revisar el sistema completo encontramos una contradicción: RF-12 dice que confirmar y cancelar transferencias lo puede hacer cualquier rol, pero el backend se lo permitía solo al encargado, y el operario veía los botones y recibía un error de permisos. Optamos por respetar RF-12. En un depósito, quien recibe la mercadería suele ser un operario, y si solo el encargado pudiera confirmar, el stock quedaría "en tránsito" más tiempo del real y podría disparar alertas de reposición falsas. El control se mantiene porque el sistema registra quién resolvió cada transferencia y cuándo (RN-14).

7.6 La API no acepta campos de más

Configuramos el backend para que rechace cualquier campo desconocido en el cuerpo de un pedido, con el código `CAMPO_NO_PERMITIDO`. Eso hace cumplir desde la propia API que la fecha de un movimiento nunca la mande el cliente (RN-41), y obliga a los formularios a mandar solo los campos del contrato (por ejemplo, nunca el `id` ni el estado en una edición).

7.7 Ajustes pedidos en la revisión del diseño

La devolución de la segunda entrega marcó casos que todavía no estaban definidos: qué pasa al dar de baja un depósito o un producto con stock, cómo evitar cantidades fraccionarias en productos que se venden por unidad y cómo unificar el criterio de baja lógica. Los cerramos antes de codificar con diez reglas nuevas (RN-38 a RN-47) y una segunda migración de la base, sin modificar la primera ya aprobada. Resolverlo en la documentación fue mucho más barato que descubrirlo después en el código.

7.8 Git en una carpeta sincronizada

Al principio, uno de los repositorios locales estaba dentro de una carpeta sincronizada con OneDrive. Al cambiar de rama, Git necesita borrar y crear carpetas, y OneDrive las bloqueaba mientras las sincronizaba: aparecían errores de "Deletion of directory failed" y quedaban archivos a medio actualizar. Lo resolvimos moviendo el repositorio a una carpeta local fuera de la sincronización y actualizando la rama principal con `git checkout -B main origin/main`, que la lleva exactamente a la versión de GitHub.

Las otras dos dificultades grandes del desarrollo, el cambio de Railway a Aiven y las pruebas de integración que no corrían, están en las secciones 10 y 9.

8. Capa analítica

La capa analítica calcula tres indicadores sobre los productos activos, a nivel global (RN-29), con parámetros que el encargado puede cambiar sin tocar el código: la ventana de consumo (30 días), los días de cobertura (30), el período de análisis (90) y los cortes del ABC (80 % y 95 %).

Punto de reposición y cantidad sugerida. El consumo promedio diario son las salidas de los últimos 30 días dividido 30; las transferencias no cuentan como consumo. Hay alerta cuando el stock disponible es menor o igual al punto de reposición.

```
\text{Punto de reposición} = \text{consumo diario} \times \text{lead time} +
\text{stock de seguridad}
```

```
\text{Cantidad sugerida} = \max(0,\ \text{PR} + \text{consumo diario} \times
\text{días de cobertura} - \text{stock disponible} - \text{en tránsito})
```

La cantidad sugerida se redondea hacia arriba, y a un número entero si el producto no admite fracciones.

Rotación. Son las salidas del período divididas por el stock promedio, que es el promedio entre el stock al principio y al final del período. Calcular el stock en una fecha pasada fue lo más delicado: una transferencia que en ese momento estaba en tránsito descuenta del total aunque después se haya confirmado o cancelado (RN-36). Lo resolvimos con una única consulta SQL que mira, para cada transferencia, si se resolvió antes o después de la fecha consultada.

Clasificación ABC. El valor de consumo de cada producto son sus salidas del período por su precio de referencia. Los productos se ordenan de mayor a menor valor y la clase se asigna con el porcentaje acumulado previo (sección 7.3): A hasta el 80 %, B hasta el 95 % y C el resto. Los productos sin consumo son siempre C.

Datos de demostración. Estos indicadores necesitan historia para mostrar algo. Como la API asigna siempre la fecha actual, agregamos un generador que, con el perfil `demo`, carga un catálogo de 15 productos, 7 proveedores y 3 depósitos con 120 días de compras, ventas, roturas y transferencias en sus tres estados. La simulación respeta las reglas de negocio (nunca deja stock negativo) y está pensada para que el ABC muestre la forma típica de Pareto, con pocos productos concentrando la mayor parte del valor, y para que algunos productos terminen en alerta de reposición. Solo corre si la base no tiene movimientos, así que no duplica datos.

9. Pruebas y validación

Concentramos las pruebas donde un error pasa desapercibido: los cálculos analíticos y las reglas de negocio. Un cálculo de rotación mal hecho no rompe ninguna pantalla, pero lleva a comprar de más o de menos.

Usamos dos niveles de pruebas:

- **Unitarias (JUnit 5 y Mockito)** sobre la capa Service, con los repositorios simulados. Cubren los casos de borde que definen las reglas: proveedor con tiempo de entrega 0, producto sin salidas,

cantidad sugerida que daría negativa, rotación con stock promedio 0, ABC con catálogo vacío, con un solo producto o con el producto que cruza un corte, y validaciones de los parámetros. Los cálculos que dependen de la fecha se prueban con un reloj fijo.

- **De integración (Testcontainers y MockMvc)** contra un MySQL 8 real, que se levanta solo en un contenedor. Verifican lo que una prueba unitaria no puede: las consultas SQL sobre las vistas de stock, el bloqueo ante operaciones simultáneas, el ciclo completo de una transferencia, el stock en una fecha pasada con transferencias en distintos estados, y los permisos de cada rol (401 sin sesión, 403 con el rol equivocado).

La suite tiene **196 pruebas automatizadas**: 160 unitarias y 36 de integración, repartidas en cinco clases que trabajan contra MySQL real. Todas pasan, sin fallas ni pruebas salteadas (`Tests run: 196, Failures: 0, Errors: 0, Skipped: 0`), y la ejecución completa con `mvn clean test` tarda menos de un minuto.

Cobertura. La meta comprometida era al menos 80 % de cobertura de líneas en la capa Service (RNF-07). El reporte de JaCoCo da **99 % de líneas cubiertas en la capa Service** (530 de 535 líneas), y ningún servicio queda por debajo de la meta:

Paquete	Líneas cubiertas
`catalogo.service`	100 % (120 de 120)
`deposito.service`	100 % (33 de 33)
`stock.service`	100 % (34 de 34)
`parametro.service`	100 % (48 de 48)
`dashboard.service`	100 % (33 de 33)
`analitica.service`	99 % (135 de 136)
`usuario.service`	98 % (45 de 46)
`movimiento.service`	96 % (82 de 85)

Sobre todo el backend, la cobertura de líneas es del 82 % (911 de 1.112). Lo que queda sin cubrir está concentrado en dos lugares que no tienen lógica de negocio: el generador de datos de demostración, que solo corre con el perfil `demo`, y algunos controllers de catálogo y depósitos, que solo derivan el pedido al servicio (las reglas que aplican están probadas en sus servicios).

Estabilizar las pruebas de integración. Al medir la cobertura descubrimos tres problemas que no eran errores del sistema, sino del entorno de pruebas, y que conviene contar porque cambiaban el resultado sin avisar:

- **Pruebas salteadas en silencio.** Si Docker no está disponible, las pruebas de integración se omiten en lugar de fallar. Con una versión reciente de Docker Desktop, la librería Testcontainers no lograba conectarse, las 36 pruebas se salteaban y la cobertura de algunos servicios aparecía en 0 %, aunque el resultado general decía BUILD SUCCESS. Desde entonces, antes de medir la cobertura verificamos que el resumen diga `Skipped: 0`.

- **Una prueba que dependía del orden.** La que comprobaba que existen las vistas de stock lo hacía exigiendo que estuvieran vacías. Como todas las pruebas de integración comparten la misma base, pasaba o fallaba según qué clases corrían antes. La corregimos para que verifique que las vistas existen y se pueden consultar, sin importar si tienen datos.
- **Un arranque demasiado justo.** En las máquinas con disco más lento, el MySQL de prueba tardaba más de dos minutos en inicializarse y Testcontainers lo daba por caído. Lo resolvimos guardando sus datos en memoria, lo que acelera mucho el arranque, y ampliando el tiempo de espera.

El reporte se genera en `backend/target/site/jacoco/index.html` al correr `mvn clean test`.

Cómo se comprueban las reglas críticas. Cada regla importante del dominio tiene al menos una prueba automática que la verifica:

Regla	Qué se comprueba	Prueba
Stock nunca negativo (RN-06)	Una salida mayor que el stock se rechaza y no registra nada.	`MovimientoIntegrationTests.salidaConStockInsuficiente_esConflictoYNoRegistraNada`
Operaciones simultáneas (D-13)	Dos salidas al mismo tiempo sobre el mismo stock no lo dejan negativo, gracias al bloqueo de la fila.	`MovimientoIntegrationTests.dosSalidasSimultaneas_noDejanElStockNegativo`
Ciclo de una transferencia (RN-10 a RN-12)	Al confirmarla suma al destino; al cancelarla vuelve al origen; mientras tanto figura en tránsito.	`cicloDeUnaTransferenciaConfirmada_actualizaStockYTransito` y `transferenciaCancelada_devuelveElStockAlOrigen`
Resolución única (RN-13)	Una transferencia ya resuelta no se puede volver a confirmar ni cancelar.	`transferenciaResuelta_noSePuedeResolverDeNuevo`
Fecha del servidor (RN-41)	El cliente no puede mandar la fecha de un movimiento.	`laFechaNoLaPuedeMandarElCliente`
Stock en una fecha pasada (RN-36)	Con transferencias confirmadas, canceladas y pendientes, el stock en cada momento es el correcto.	`AnaliticaIntegrationTests.stockTotalAl_consideraElEstadoDeLasTransferenciasEnEseMomento_RN36`
Punto de reposición (RN-20 a RN-24)	La fórmula, el caso sin salidas, el lead time 0, la cantidad sugerida que no puede ser negativa y el redondeo.	`ReposicionServiceTest`
Corte del ABC (D-12)	El producto que cruza un corte queda en la clase superior; sin consumo es C.	`AbcServiceTest`
Permisos (RN-34, RN-35)	El operario recibe 403 en las rutas del encargado y en la capa analítica.	`SeguridadIntegrationTests` y `AnaliticaIntegrationTests`

Regla	Qué se comprueba	Prueba
Usuario desactivado (RF-02)	Pierde el acceso al instante, aunque su token siga vigente.	`usuarioDesactivadoConTokenVigente_pierdeElAccesoDeInmediato_RF02`

Prueba de punta a punta en producción. Además de las pruebas automáticas, recorrimos el sistema publicado con los dos roles siguiendo una lista de unos 50 casos, organizada en nueve secciones: ingreso y sesión, permisos, usuarios, catálogo, depósitos, movimientos y transferencias, stock, capa analítica y situaciones límite (servidor dormido, celular, pantallas sin resultados). Incluyó verificar a mano el cálculo de una alerta y comprobar que un cambio de parámetros modifica los resultados. También cada etapa incluyó un archivo de pedidos de ejemplo (`backend/http/`) para probar la API a mano, incluidos los casos de error esperados.

Validación con el caso real. Jonathan, como jefe de almacén y usuario real del problema, recorrió el sistema pensando en su trabajo diario. Su conclusión fue que le ahorraría trabajo frente a la planilla y que, sobre todo, le permitiría ver qué productos están inmovilizados para tomar medidas concretas, como ponerlos en oferta. Esa es justamente la decisión que la planilla no le permitía tomar con 600 a 700 productos. La validación se hizo con los datos de demostración, no con los datos reales de su empresa.

10. Despliegue

El sistema está publicado en tres plataformas gratuitas: la base de datos en **Aiven** (MySQL 8), el backend en **Render** (contenedor Docker) y el frontend en **Vercel**. Cada pull request integrado en `main` se vuelve a publicar solo, y la guía completa está en `docs/07-despliegue.md`.

Desplegar fue la etapa con más imprevistos, y casi todos vinieron de las plataformas, no del código:

- **Railway cerró las altas.** El diseño original usaba Railway para MySQL, pero en octubre de 2026 pausó la creación de cuentas nuevas tras un incidente de seguridad. Lo reemplazamos por Aiven, que ofrece MySQL 8 gratis y sin vencimiento. Como el backend se conecta a cualquier MySQL, el cambio no tocó el código: solo los datos de conexión, a los que hubo que sumar la conexión cifrada (`sslMode=REQUIRED`) que exige Aiven.
- **El nombre de la base.** Evitamos el guion en el nombre (`stockiq_db` en lugar de `stockiq-db`), porque en MySQL obliga a usar comillas invertidas y puede traer errores con Flyway.
- **Vercel y su protección por defecto.** El primer despliegue quedó como versión de prueba y no como producción, y las direcciones de prueba podían verlas solo los miembros del equipo. Lo resolvimos volviendo a importar el proyecto con la carpeta `frontend` como raíz y desactivando la protección, para que cualquiera pueda abrir el sistema.
- **El backend que se duerme.** El plan gratuito de Render apaga el servicio después de un rato sin uso, y el primer pedido puede tardar más de un minuto. La pantalla de login avisa cuando pasa, y para las presentaciones usamos un servicio externo que visita el chequeo de estado (`/actuator/health`) cada pocos minutos.
- **CORS.** El navegador bloquea los pedidos del frontend al backend si la dirección de Vercel no está autorizada exactamente igual (con `https` y sin barra final) en la variable `CORS_ALLOWED_ORIGINS`.

La lección principal fue la que nos anticipó la tutora: desplegar temprano. Todos estos problemas aparecieron con el sistema a medio construir, y los resolvimos con tiempo, sin la presión de la semana de entrega.

11. Resultados, conclusiones y trabajo futuro

Cumplimos el objetivo del trabajo: StockIQ registra cada movimiento con trazabilidad completa, calcula el stock sin guardarlo y propone cuándo y cuánto reponer con datos reales de consumo, y está publicado y funcionando.

Qué cambia respecto de la planilla. Frente a la situación que dio origen al proyecto, el sistema aporta estos cambios concretos:

- **El stock deja de cargarse a mano:** se calcula a partir de los movimientos, así que no puede quedar desactualizado ni contradecir el historial.
- **Cada movimiento queda registrado** con quién lo hizo, cuándo y por qué, y no se puede editar ni borrar.
- **La mercadería en camino entre depósitos es visible,** y solo suma al destino cuando alguien confirma que llegó.
- **La decisión de reponer deja de ser "a ojo":** el sistema calcula el punto de reposición con el consumo real y el tiempo de entrega del proveedor, y sugiere cuánto pedir.
- **Los productos de baja rotación y los paralizados se identifican solos,** aun con cientos de productos: el dashboard los ordena por rotación y muestra cuánta plata tienen inmovilizada.
- **La carga manual tiene controles:** cuando una entrada se registra a partir de un remito en papel, el sistema no acepta productos inactivos, cantidades con decimales en productos por unidad ni salidas mayores que el stock, y deja registrado quién la cargó. La carga sigue siendo manual (ver trabajo futuro), pero los errores más comunes se frenan en el momento.

Más allá del sistema, nos llevamos tres aprendizajes:

1. **Cerrar el análisis antes de codificar ahorra trabajo.** Las reglas que la tutora nos pidió definir en la revisión del diseño (bajas con stock, cantidades fraccionarias, zona horaria) aparecieron después en el código exactamente como lo había anticipado, pero ya estaban resueltas.
2. **En un trabajo en equipo, el contrato vale tanto como el código.** El único error de integración importante vino de un documento ambiguo, y desapareció cuando empezamos a escribir el JSON exacto de cada endpoint antes de programarlo.
3. **Desplegar temprano es parte del desarrollo.** Los problemas de las plataformas no dependen de la calidad del código, y conviene encontrarlos con margen.

Limitaciones conocidas. Todo lo descripto en este informe está implementado y probado; lo que sigue son límites del diseño actual o del entorno:

- **Todas las salidas cuentan como consumo,** incluidas las de ajuste, que pueden sobrestimarlo mientras estén dentro de la ventana de cálculo (documentado en `docs/02-reglas-de-negocio.md`).
- **El ABC valoriza el período con el precio actual** de cada producto, así que un cambio de precio recalcula el ranking como si siempre hubiera tenido ese precio.

- **El punto de reposición y el ABC se calculan a nivel global,** sumando todos los depósitos, y no por depósito.
- **Cada producto tiene un solo proveedor.**
- **El stock de seguridad lo carga el encargado;** no se calcula a partir de la variabilidad de la demanda.
- **El entorno publicado usa planes gratuitos:** el backend se apaga después de un rato sin uso (el primer ingreso puede tardar hasta un minuto), y su capacidad de procesamiento es limitada, por lo que el dashboard puede tardar algunos segundos. Los datos que se ven son de demostración, generados con 120 días de historia.

Como trabajo futuro, sin implementar todavía, proponemos:

- Clasificar los motivos de salida e indicar cuáles cuentan como consumo.
- Guardar el precio en cada salida, para valorizar el ABC con el precio de ese momento.
- Permitir varios proveedores por producto, con su propio tiempo de entrega.
- Calcular el punto de reposición y el ABC por depósito, y no solo a nivel global.
- Importar las entradas de fábrica desde un archivo (por ejemplo, el remito en formato digital), para evitar la carga manual a partir del papel.
- Gestionar devoluciones y enviar las alertas de reposición por correo.
- Calcular el stock de seguridad según la variabilidad de la demanda, con la fórmula clásica: Z × desvío estándar del consumo diario × √lead time, donde Z depende del nivel de servicio buscado (por ejemplo, 1,65 para un 95 %).

Video de presentación: https://youtu.be/QdCD1V_JVE4